

Assignment 2- Case Study

Stage 3 Lab Activities

SmartCare Domain Modelling

AI OFF -> AI ON -> COMPARE -> VERIFY | 1 hour

A - Requirements Review

Highlight nouns, verbs and business rules in SmartCare v0.2.

Nouns: Patients, practitioner, appointment, clinic

Verbs: manage patients, manage practitioners, manage appointments, book appointment, cancel appointment, maintain records.

Business rules:

- . An appointment must be associated with a patient.
- . An appointment must be associated with a practitioner.
- . An appointment has a scheduled status when created.
- . An appointment can be cancelled.

B - Candidate Classes

Record candidate concepts, supporting requirements, state and behaviour.

Candidate concept	Supporting requirements	State	Behaviour
Patient	Managing patient information and history	Patient details and history	View patient data
Practitioner	Managing practitioners	Practitioners details	View practitioner data
Appointment	Manage, book and cancel appointment	Appointment time and date	Schedule/ cancel appointment
Clinic	Manages everything	Clinic information	Manage details and records

C - CRC Cards

Create CRC cards for Patient, Practitioner and Appointment.

Patient

- Class: Patient
- Responsibilities: Patient information.
- Collaborators: Appointment

Practitioner

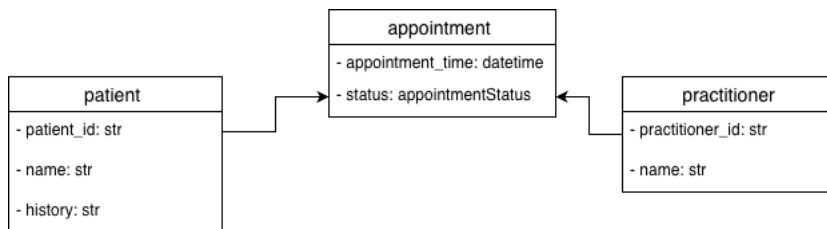
- Class: Practitioner
- Responsibilities: Practitioner details
- Collaborators: Appointment

Appointment

- Class: Appointment
- Responsibilities: Appointment time, patient/practitioner association, status, cancellation
- Collaborators: Patient, Practitioner

D - UML Model

Draw classes, attributes, operations, associations and multiplicities.



E - AI Design Review

Ask AI to suggest classes and relationships using only confirmed requirements; require supporting requirement IDs.

The AI design review used only the confirmed SmartCare requirements to suggest suitable classes and relationships. It identified **Patient**, **Practitioner**, and **Appointment** as the main classes because the requirements state that the system manages patients, practitioners, and appointments. The review also suggested an **AppointmentStatus** enumeration with the states **SCHEDULED** and **CANCELLED**, supported by the requirements for appointment status and cancellation. The **Appointment** class includes the appointment time and a `cancel()` operation. Relationships were identified between Patient–Appointment and Practitioner–Appointment because every appointment is associated with a patient and practitioner. Each suggestion was linked to its supporting requirement ID. No additional classes, features, or assumptions were introduced without confirmed requirements.

F - Compare and Decide

Record at least one accepted, modified and rejected AI suggestion.

Accepted: using patient, practitioner, appointment as a class

Modified: using AppointmentStatus as a class

Rejected: unnecessary and irrelevant suggestions were rejected like notification manager

G - Python Skeletons

Create simple Patient, Practitioner and Appointment class skeletons.

```
class AppointmentStatus:
    Scheduled= "Scheduled"
    Cancelled= "Cancelled"
class Patient:
    def __init__(self, name: str):
        self.name= name
class Practitioner:
    def __init__(self, name: str):
        self.name= name
class Appointment:
    def __init__(
        self,
        patient: Patient,
        practitioner: Practitioner,
        appointment_time: str
    ):
        Self.patient = patient
        Self.practitioner = practitioner
        Self.appointment_time = appointment_time
        Self.status = AppointmentStatus.Scheduled
Def cancel(self)-> none:
```

H - Consistency Check

Check model-code consistency; do not implement full behaviour yet.

- The consistency has been checked and it is functional

Reflection

What modelling decision was hardest? Where did AI over-design? What evidence supported your final choices?

- The hardest decision was to decide which class and relationships were necessary for the Smartcare system without unnecessary features.
- The AI initially started making it advanced with its vast capabilities adding extra features.
- The final choices were supported by the confirmed requirements, the approved UML, and the consistency check between the UML and Python skeletons.